CHUKA UNIVERSITY

School of Computing and Informatics

DATABASE SYSTEMS

Concurrency Control — Group Assignment

Course Unit:	Database Systems
Topic:	Concurrency Control Protocols
Submission To:	kotieno@chuka.ac.ke
Deadline:	25th March 2025
Format:	Typed — No Handwriting Accepted

NB: Everything must be typed. No handwriting will be accepted.

PART A — Concurrent Transactions (≥ 10, with ≥ 6 Updates)

Database Context

The following transactions operate on a school fee management database with tables: **Student(StudentID, Balance, Grade)**, **Fees(FeeID, StudentID, Amount, Term)**, and **Account(AccID, TotalFunds)**. Items referenced: **A** = Student.Balance (S001), **B** = Account.TotalFunds, **C** = Student.Balance (S002), **D** = Fees.Amount (F001), **E** = Student.Grade (S001).

Txn	Operations	Type	Description
T1	`READ(A); A := A - 5000; WRITE(A); READ(B); B := B + 5000; WRITE(B)`	UPDATE	Fee payment by Student S001
T2	`READ(C); C := C - 3000; WRITE(C); READ(B); B := B + 3000; WRITE(B)`	UPDATE	Fee payment by Student S002
T3	`READ(A); READ(C); PRINT(A+C)`	READ-ONLY	Generate total outstanding balance report
T4	`READ(D); D := D * 1.10; WRITE(D)`	UPDATE	Apply 10% fee increment for Term 3
T5	`READ(B); PRINT(B)`	READ-ONLY	Audit — read total funds in account
T6	`READ(A); A := A + 2000; WRITE(A)`	UPDATE	Reverse partial fee reversal for S001
T7	`READ(E); E := 'Grade 5'; WRITE(E)`	UPDATE	Promote student S001 to Grade 5
T8	`READ(C); C := C - 1500; WRITE(C); READ(B); B := B + 1500; WRITE(B)`	UPDATE	Activity fee deduction for S002
T9	`READ(A); READ(B); READ(C); PRINT(A,B,C)`	READ-ONLY	Full snapshot read for end-of-term report
T10	`READ(D); D := D - 500; WRITE(D); READ(B); B := B - 500; WRITE(B)`	UPDATE	Fee correction / discount applied to F001

Transactions T1, T2, T4, T6, T7, T8, T10 perform WRITE operations (7 updating transactions ≥ 6 required). T3, T5, T9 are read-only. All ten transactions may execute concurrently.

PART B — Schedules Under the Four Lock-Based Protocols

Four lock-based concurrency control protocols are examined. For clarity, a representative subset of transactions (T1, T2, T3, T4) is scheduled under each protocol, as these cover all conflict patterns: read–write, write–write, and read–read.

B1 — Simple (Binary) Locking Protocol

Each transaction must acquire a single binary LOCK on a data item before accessing it and release it immediately after use. There is no distinction between read and write locks.

Choices Made:

- Only one type of lock exists (binary lock). Any transaction holding the lock blocks all others.
- T1 acquires lock on A, writes, releases; then locks B, writes, releases.
- T2 must wait for T1 to release lock on B before it can proceed — serialized on shared item B.
- T3 (read-only) still must acquire binary locks and waits behind write transactions.
- This protocol is simple but overly restrictive — reads block reads unnecessarily.

Step	T1	T2	T3	T4	Lock Table
1	LOCK(A)				A: T1
2	READ(A)				A: T1
3	WRITE(A)				A: T1
4	UNLOCK(A)				A: free
5	LOCK(B)	wait(B)			B: T1
6	READ(B)	wait(B)			B: T1
7	WRITE(B)	wait(B)			B: T1
8	UNLOCK(B)	LOCK(B)			B: T2
9		READ(C)	wait(A)		B:T2, A:free
10		WRITE(C)			B:T2, C:T2
11		WRITE(B)			B:T2
12		UNLOCK(B)	LOCK(A)		A:T3, B:free
13		UNLOCK(C)	READ(A)	LOCK(D)	A:T3, D:T4
14			READ(C)	READ(D)	A:T3,C:T3,D:T4
15			UNLOCK(A)	WRITE(D)	C:T3,D:T4
16			UNLOCK(C)	UNLOCK(D)	free

Outcome: Schedule is serializable (equivalent to T1 → T2 → T3 → T4). Deadlock-free here because items are accessed in a consistent order.

B2 — Shared / Exclusive (Read/Write) Lock Protocol

Two lock modes: **S-lock** (shared, for reads) and **X-lock** (exclusive, for writes). Multiple transactions may hold S-locks simultaneously; an X-lock requires exclusive access.

Choices Made:

- T3 (read-only) acquires S-locks, allowing T1 and T3 to read A concurrently.
- T1 upgrades to X-lock on A before writing — T3 must wait for the upgrade.
- T2 and T1 conflict on B (both write) — T2 waits for T1's X-lock on B to release.
- More concurrency than simple locking: T3's read of A overlaps with T1's read of A.
- Compatible matrix: S+S = allowed; S+X = blocked; X+X = blocked.

Step	T1	T2	T3	T4	Locks Held
1	S-lock(A)		S-lock(A)		A: S(T1), S(T3)
2	READ(A)		READ(A)		A: S(T1), S(T3)
3	Upgrade X(A)		wait(A)		A: X(T1)
4	WRITE(A)		wait(A)		A: X(T1)
5	X-lock(B)	wait(B)	wait(A)		A:X(T1), B:X(T1)
6	WRITE(B)	wait(B)	wait(A)		A:X(T1), B:X(T1)
7	UNLOCK(A,B)	X-lock(C)	S-lock(A)	X-lock(D)	C:X(T2),A:S(T3),D:X(T4)
8		X-lock(B)	READ(A)	READ(D)	C:X(T2),B:X(T2),A:S(T3),D:X(T4)
9		WRITE(C)	S-lock(C)	WRITE(D)	C:X(T2),B:X(T2),A:S(T3),D:X(T4)
10		WRITE(B)	wait(C)	UNLOCK(D)	C:X(T2),B:X(T2),A:S(T3)
11		UNLOCK(B,C)	S-lock(C)		C:S(T3),A:S(T3)
12			READ(C)		C:S(T3),A:S(T3)
13			UNLOCK(A,C)		free

Outcome: Greater concurrency than binary locking. T1 and T3 read A simultaneously. Schedule is conflict-serializable. No deadlock in this execution order.

B3 — Two-Phase Locking (2PL) Protocol

Each transaction has two distinct phases: **Growing Phase** — locks are acquired, none released; **Shrinking Phase** — locks are released, none acquired. 2PL guarantees conflict serializability.

Choices Made:

- All lock acquisitions happen at the start of each transaction (growing phase).
- No lock is released until the transaction enters its shrinking phase (after last operation).
- T1 holds locks on A and B throughout its entire execution before releasing both.
- T2 waits for T1 to fully release locks — ensures no dirty reads or lost updates.
- The lock point (end of growing phase) determines serialization order: T1 → T2 → T3 → T4.
- 2PL can cause deadlocks (e.g., T1 holds A and waits for B; T2 holds B and waits for A).

Step	T1	T2	T3	T4	Phase / Locks
1	X-lock(A)				T1 growing: A
2	X-lock(B)				T1 growing: A,B
3	READ(A)				T1 @ lock point
4	WRITE(A)				T1 shrinking soon
5	READ(B)	S-lock(A)			T2 growing; T1 still holds B
6	WRITE(B)	wait(B)			T2 blocked on B
7	UNLOCK(A)	wait(B)			T1 shrinking; A released
8	UNLOCK(B)	X-lock(B)			T2 gets B; growing: A,B,C
9		X-lock(C)			T2 @ lock point
10		READ(C)	S-lock(A)	X-lock(D)	T3 growing; T4 growing
11		WRITE(C)	S-lock(C)	READ(D)	T3 @ lock pt; T4 @ lock pt
12		WRITE(B)	READ(A)	WRITE(D)	T3,T4 shrinking phase
13		UNLOCK(B,C)	READ(C)	UNLOCK(D)	T2 done; T4 done
14			UNLOCK(A,C)		T3 done

Outcome: Conflict-serializable (order: T1→T2→T3→T4). 2PL guarantees serializability but may cause deadlocks in more complex interleavings. A deadlock-detection mechanism or conservative 2PL (acquire all locks upfront) would address this.

B4 — Strict Two-Phase Locking (Strict 2PL)

An extension of 2PL: all **exclusive (write) locks** are held until the transaction **commits or aborts**. This prevents dirty reads and enables cascadeless recovery.

Choices Made:

- X-locks are never released before COMMIT — other transactions cannot read uncommitted data.
- T2 must wait for T1 to COMMIT before reading/writing B, preventing dirty reads.
- S-locks may be released earlier (after read), but X-locks are released only at commit.
- Rigorous 2PL (a stricter variant) would hold ALL locks until commit — we use Strict 2PL here.
- Ensures recoverability: if T1 aborts, T2 (which waited) is unaffected.
- More blocking than regular 2PL but provides stronger safety guarantees.

Step	T1	T2	T3	T4	Locks / Notes
1	X-lock(A,B)				T1 holds X(A),X(B) until commit
2	READ(A)	wait(A,B)			T2 blocked — X-locks held
3	WRITE(A)	wait(A,B)			T2 still waiting
4	READ(B)	wait(A,B)	S-lock(A) blocked		T3 also blocked
5	WRITE(B)	wait	blocked	X-lock(D)	T4 independent — gets D
6	COMMIT T1			READ(D)	T1 commits; releases X(A),X(B)
7		X-lock(B,C)	S-lock(A)	WRITE(D)	T2 and T3 proceed
8		READ(C)	READ(A)	COMMIT T4	T4 done
9		WRITE(C,B)	S-lock(C)		T3 waits for C from T2
10		COMMIT T2	wait(C)		T2 commits; releases C
11			READ(C)		T3 reads C safely
12			COMMIT T3		All done; no dirty reads

Outcome: Strictly serializable and recoverable. No dirty reads. Serialization order: T1 → T4 (parallel) → T2 → T3. T4 operates on D independently and commits before T2 even starts — true concurrency.

PART C — Schedule Under Timestamp-Based Protocol

In the Timestamp Ordering (TSO) protocol, each transaction T_i is assigned a unique timestamp $TS(T_i)$ at the moment it starts. Each data item X maintains two timestamps: **R-timestamp(X)** — the largest timestamp of any transaction that successfully read X, and **W-timestamp(X)** — the largest timestamp of any transaction that successfully wrote X. No locks are used; conflicts are resolved by timestamp comparison.

C1 — Timestamp Assignment

Transaction	Timestamp	Start Order	Type
T1	TS=1	Earliest	UPDATE (A, B)
T2	TS=2	2nd	UPDATE (C, B)
T3	TS=3	3rd	READ-ONLY (A, C)
T4	TS=4	4th	UPDATE (D)
T5	TS=5	5th	READ-ONLY (B)
T6	TS=6	6th	UPDATE (A)
T7	TS=7	7th	UPDATE (E)
T8	TS=8	8th	UPDATE (C, B)
T9	TS=9	9th	READ-ONLY (A,B,C)
T10	TS=10	Latest	UPDATE (D, B)

C2 — TSO Rules Applied

Read Rule: T_i issues READ(X):

- If TS(Ti) < W-timestamp(X) → transaction is too late; ROLLBACK Ti, restart with new timestamp.
- If TS(Ti) ≥ W-timestamp(X) → READ is allowed; update R-timestamp(X) = max(R-ts(X), TS(Ti)).

Write Rule: T_i issues WRITE(X):

- If TS(Ti) < R-timestamp(X) → write rejected (a later transaction already read X); ROLLBACK Ti.
- If TS(Ti) < W-timestamp(X) → obsolete write (Thomas Write Rule: safely ignore); continue.
- Otherwise → WRITE is allowed; update W-timestamp(X) = TS(Ti).

C3 — Timestamp Schedule Execution

Initial state: All R-ts and W-ts = 0 for all items.

Step	Operation	TS(Ti)	W-ts(X)	R-ts(X)	Decision	New Timestamps
1	T1: READ(A)	1	0	0	ALLOW (1≥0)	R-ts(A)=1
2	T1: WRITE(A)	1	0	1	ALLOW (1≥0, 1≥1)	W-ts(A)=1
3	T1: READ(B)	1	0	0	ALLOW	R-ts(B)=1
4	T1: WRITE(B)	1	0	1	ALLOW	W-ts(B)=1
5	T2: READ(C)	2	0	0	ALLOW	R-ts(C)=2

Step	Operation	TS(Ti)	W-ts(X)	R-ts(X)	Decision	New Timestamps
6	T2: WRITE(C)	2	0	2	ALLOW	W-ts(C)=2
7	T2: READ(B)	2	1	1	ALLOW (2≥1)	R-ts(B)=2
8	T2: WRITE(B)	2	1	2	ALLOW (2≥2)	W-ts(B)=2
9	T3: READ(A)	3	1	0	ALLOW (3≥1)	R-ts(A)=3
10	T3: READ(C)	3	2	2	ALLOW (3≥2)	R-ts(C)=3
11	T4: WRITE(D)	4	0	0	ALLOW	W-ts(D)=4
12	T5: READ(B)	5	2	2	ALLOW (5≥2)	R-ts(B)=5
13	T6: READ(A)	6	1	3	ALLOW (6≥1)	R-ts(A)=6
14	T6: WRITE(A)	6	1	6	ALLOW (6≥6)	W-ts(A)=6
15	T7: WRITE(E)	7	0	0	ALLOW	W-ts(E)=7
16	T8: READ(C)	8	2	3	ALLOW (8≥2)	R-ts(C)=8
17	T8: WRITE(C)	8	2	8	ALLOW (8≥8)	W-ts(C)=8
18	T8: READ(B)	8	2	5	ALLOW (8≥2)	R-ts(B)=8
19	T8: WRITE(B)	8	2	8	ALLOW (8≥8)	W-ts(B)=8
20	T9: READ(A)	9	6	6	ALLOW (9≥6)	R-ts(A)=9
21	T9: READ(B)	9	8	8	ALLOW (9≥8)	R-ts(B)=9
22	T9: READ(C)	9	8	8	ALLOW (9≥8)	R-ts(C)=9
23	T10: WRITE(D)	10	4	0	ALLOW (10≥0)	W-ts(D)=10
24	T10: READ(B)	10	8	9	ALLOW (10≥8)	R-ts(B)=10
25	T10: WRITE(B)	10	8	10	ALLOW (10≥10)	W-ts(B)=10

C4 — Explanation of Choices & Outcomes

- No rollbacks occurred in this schedule because transactions are submitted in strict timestamp order (T1 first, T10 last), meaning no transaction ever attempts to read or write data that a later transaction has already modified.
- The schedule is equivalent to the serial order T1 → T2 → T3 → T4 → T5 → T6 → T7 → T8 → T9 → T10 — the timestamp order itself defines serializability.
- Thomas Write Rule was not triggered here, but if T3 (TS=3) had tried to WRITE(A) after T6 (TS=6) already wrote it (W-ts(A)=6), the write would be silently ignored as obsolete rather than causing a rollback.
- Timestamp-based protocols are deadlock-free by design: transactions never wait for locks. Conflicts are resolved immediately by rollback or ignore.
- The main disadvantage is starvation: a transaction that repeatedly gets a lower timestamp than competing transactions may be continually rolled back and restarted.
- In a realistic concurrent submission scenario, some transactions with conflicting timestamps would be rolled back and restarted with new (larger) timestamps.

PART D — Submission Information

One member of the group is required to submit this assignment on behalf of the entire group.

Item	Detail
Submit To	kotieno@chuka.ac.ke
Deadline	25th March 2025
Format	PDF — typed, no handwriting
Subject Line (suggested)	DB Concurrency Control Assignment — Group [Your Group Number]
Content	This document covers Parts A, B, C, and D as required

© Designed by Kelvin Mutinda 2025 · Chuka University · Database Systems Group Assignment